VOUGNE FROID ALIS

RECURSION & PROBLEM SOLVING

VOUGNE FROID ALIS

WHAT IS RECURSION?

Definition: The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

VOUGNE FROID ALIS

THE RULES OF RECURSION

1. The Base Case (The Brake Pedal)
The condition where the function stops calling itself.
Example: "If the number is 1, stop."

2. The Recursive Case (The Gas Pedal)
The part where the function calls itself with a smaller input.
Example: "Return n * factorial(n-1)."

VOUGNE FROID ALIS

LET’S VISUALIZE!!

VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

1. What is the simplest possible input

```
sum(0) = 0
sum(1) = 1
sum(4) = 1+2+3+4      = 10
sum(5) = 1+2+3+4+5    = 15
```

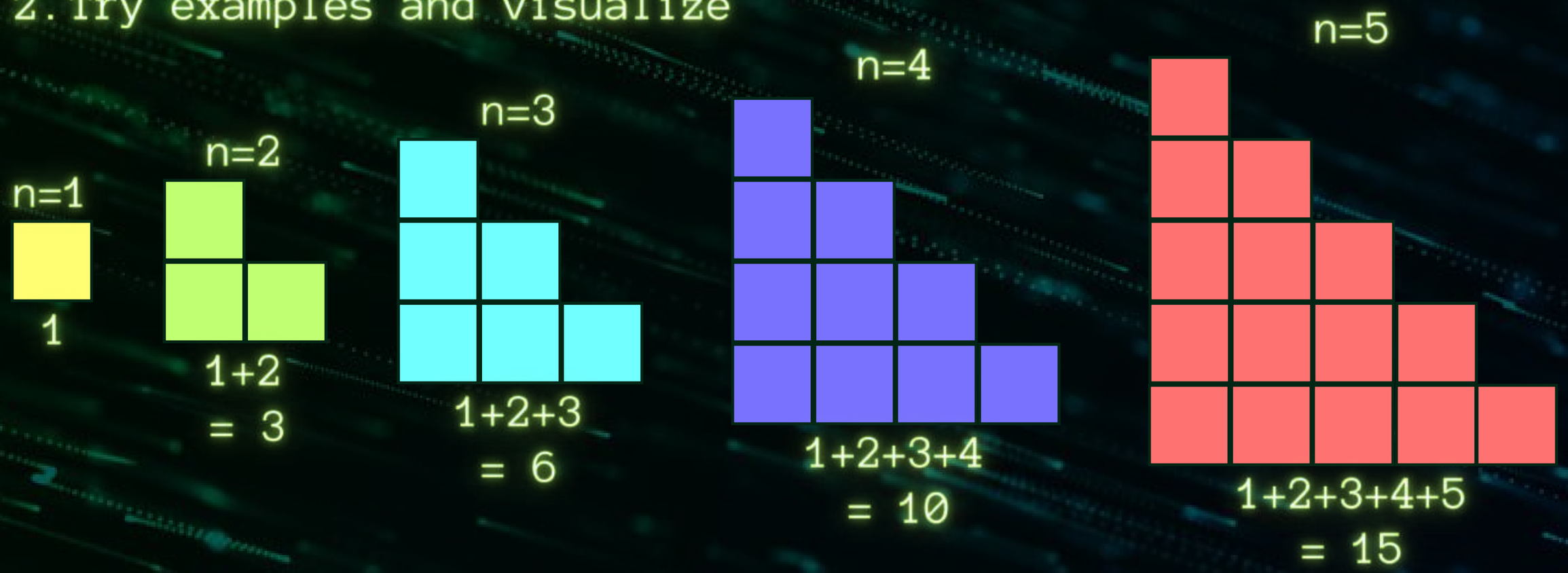
Lets take a look at the iterative approach

VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

2. Try examples and visualize

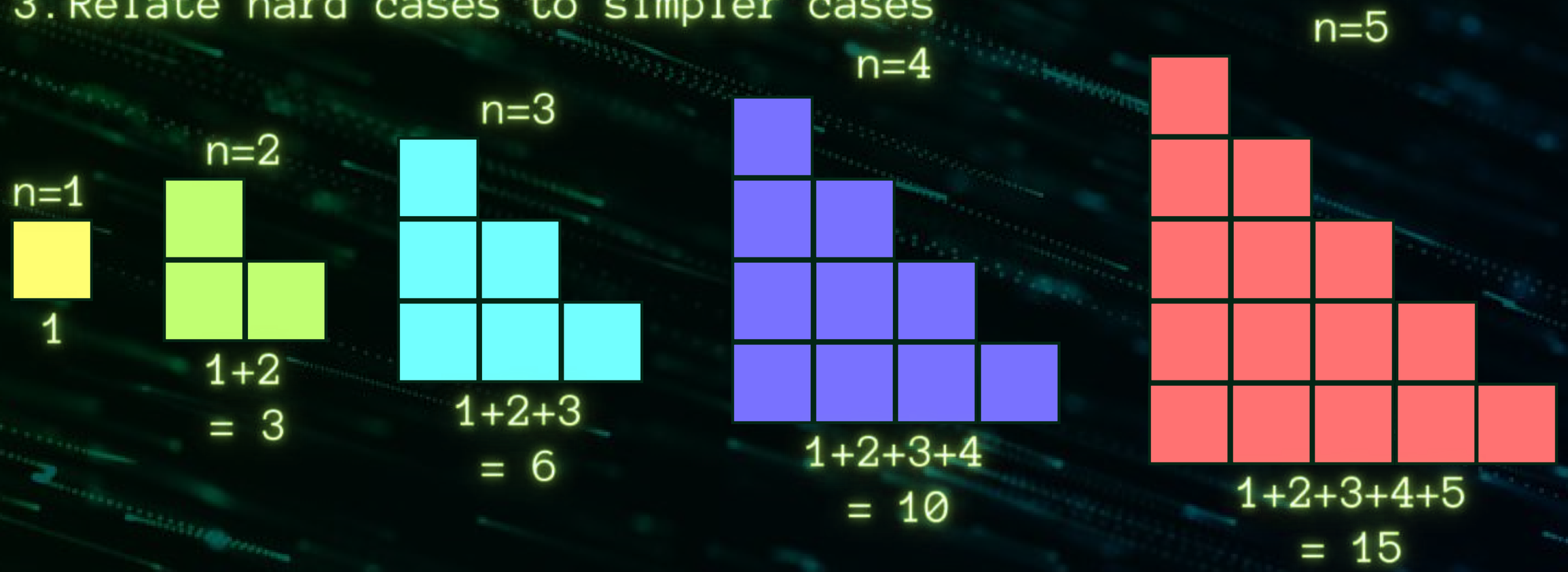


VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

3. Relate hard cases to simpler cases

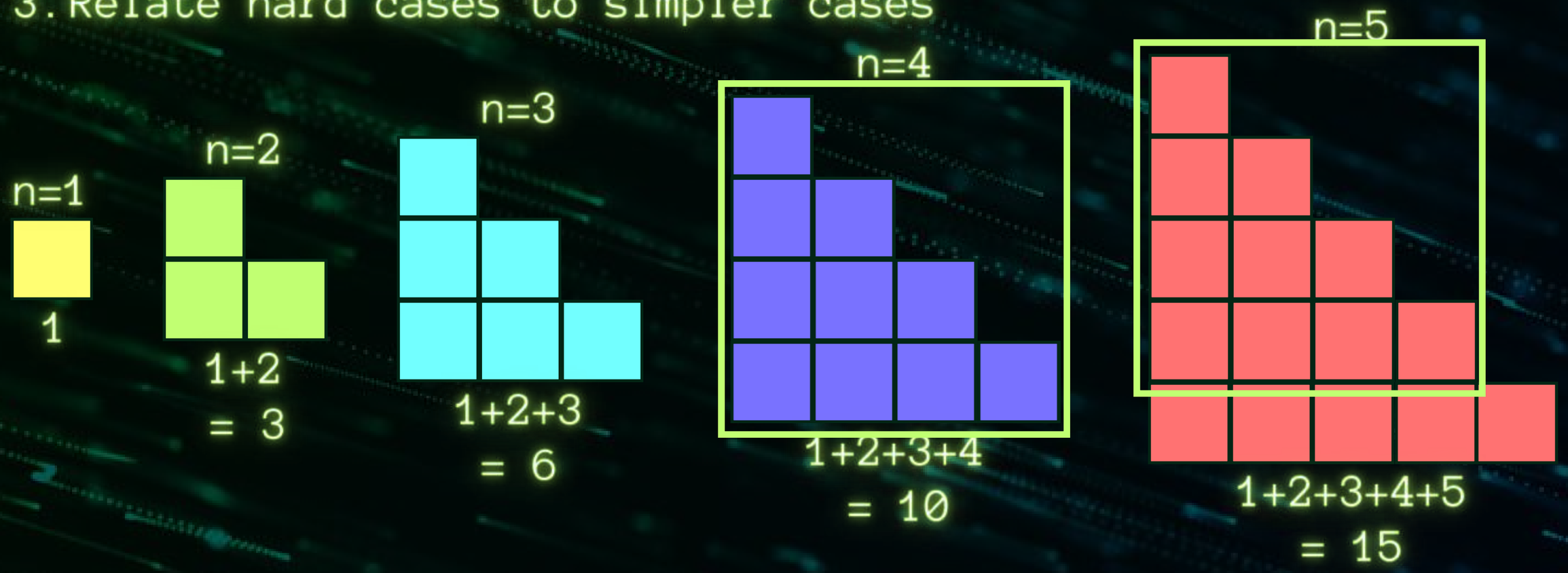


VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

3. Relate hard cases to simpler cases

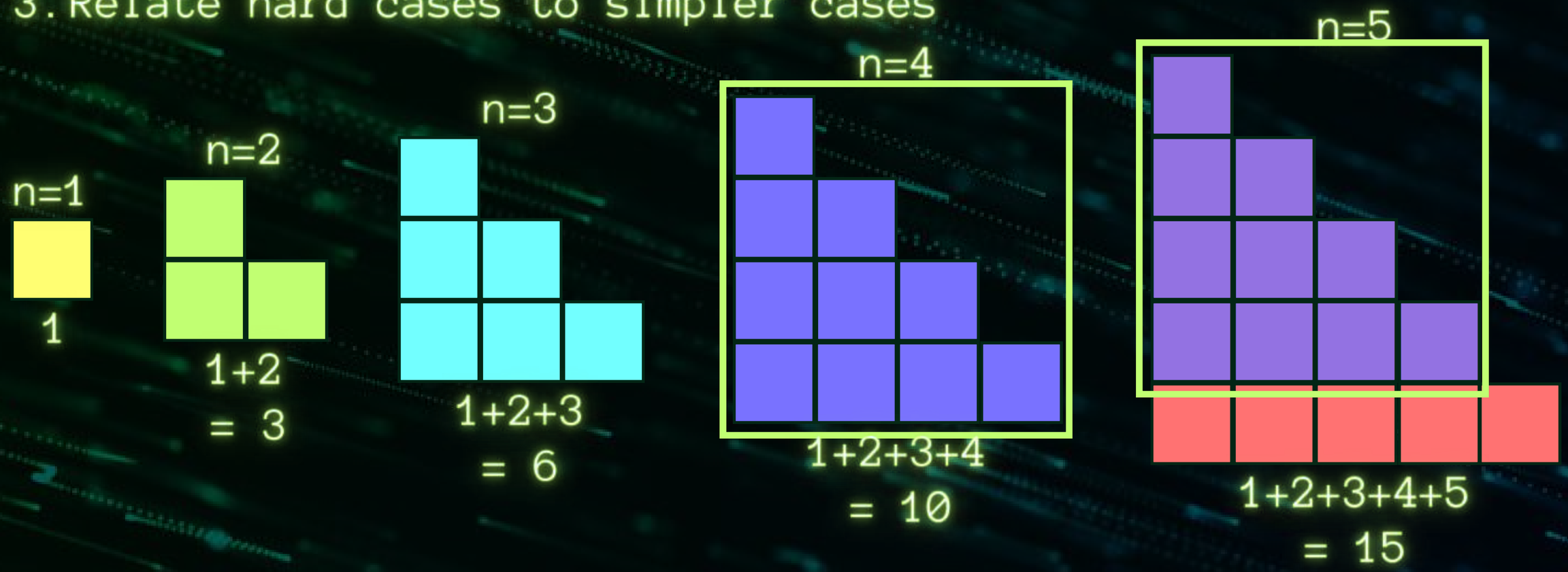


VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

3. Relate hard cases to simpler cases

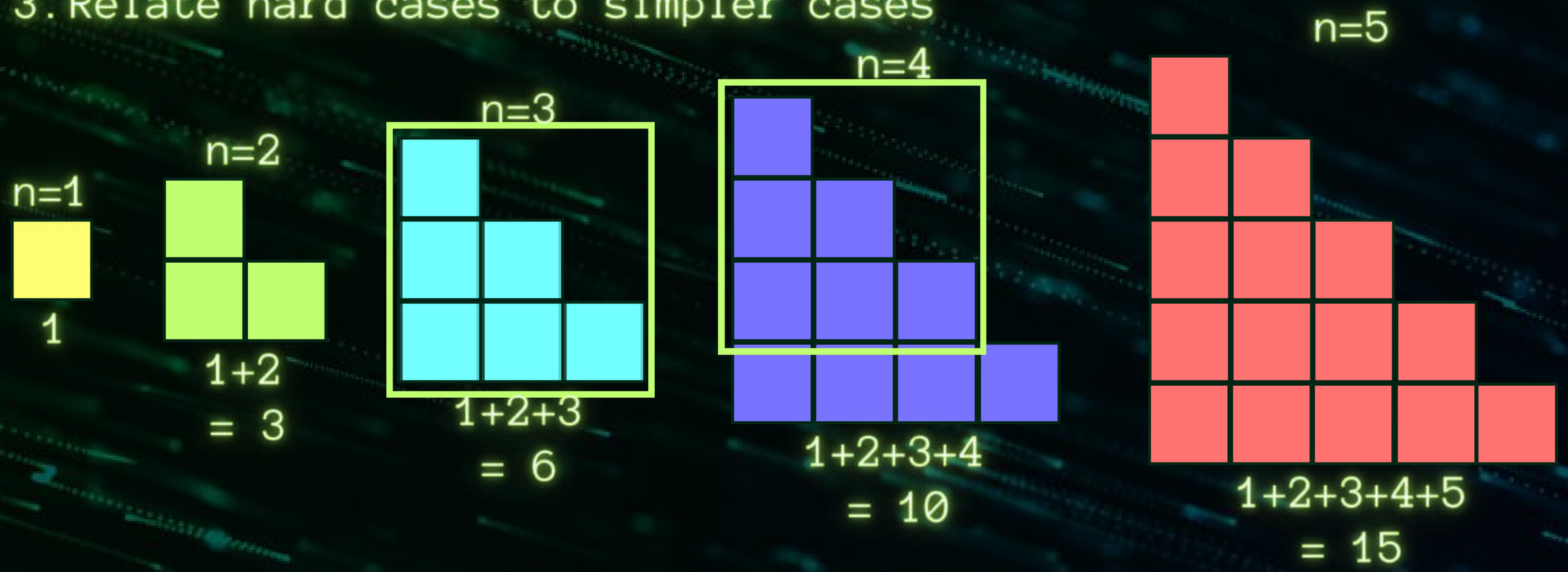


VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

3. Relate hard cases to simpler cases

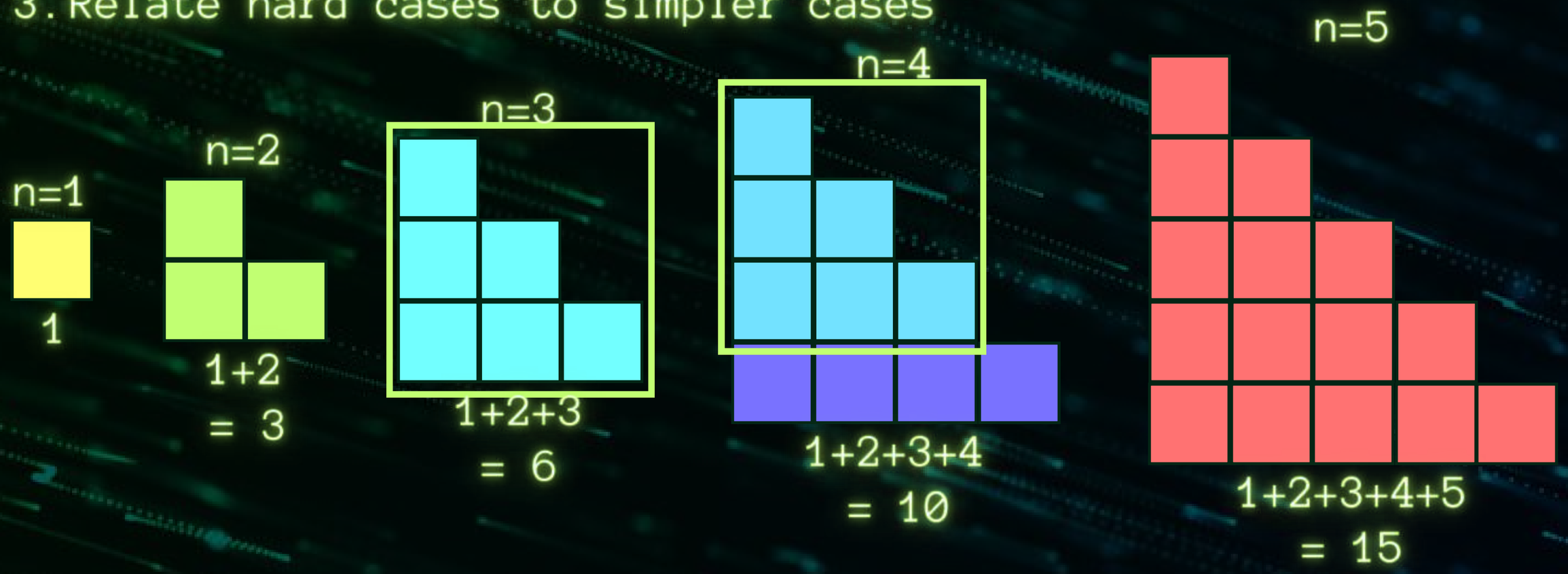


VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

3. Relate hard cases to simpler cases



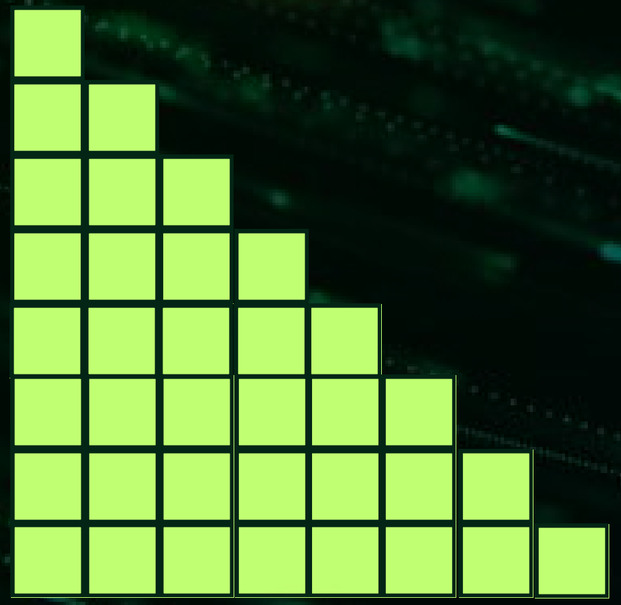
VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

4. Generalize the pattern

n=k



=

k

1+2+...+k

sum(k)

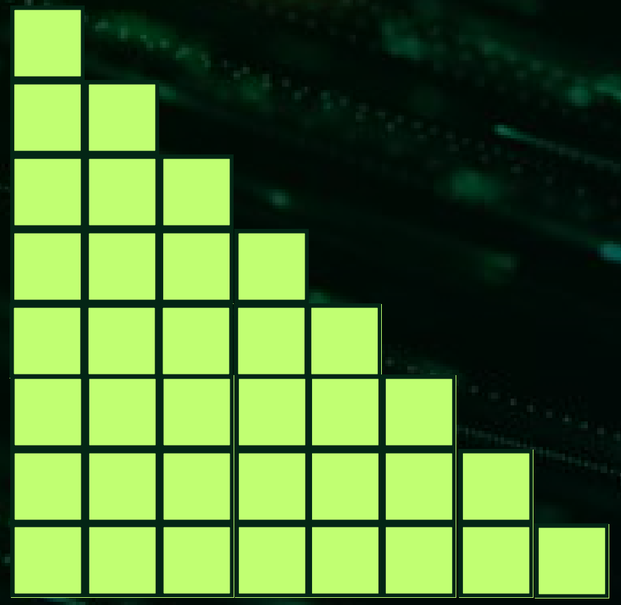
VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

4. Generalize the pattern

n=k



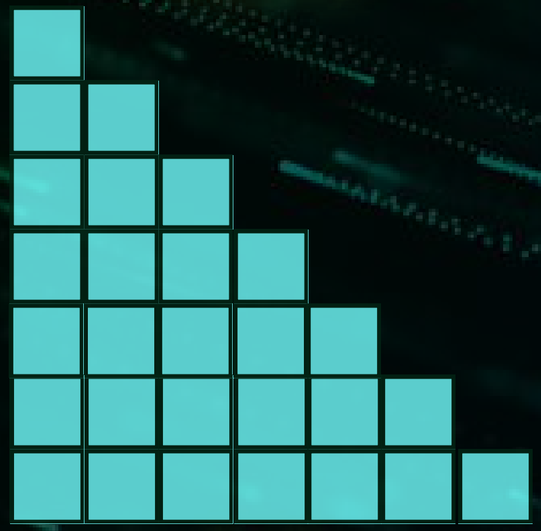
k

1+2+...+k

sum(k)

=

n=k-1



k-1

1+2+...+(k-1)

sum(k-1)

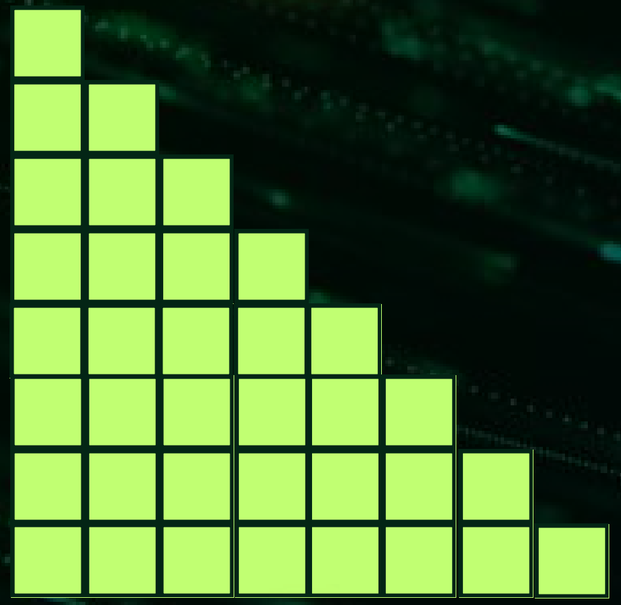
VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

4. Generalize the pattern

n=k



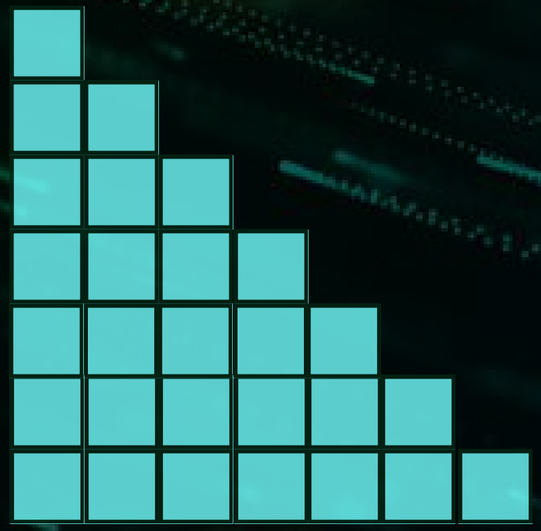
k

1+2+...+k

sum(k)

=

n=k-1



k-1

1+2+...+(k-1)

sum(k-1)

+

k

VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

5. Write the code by combining the recursive pattern with the base case

$$sum(k) = sum(k-1) + k$$

VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

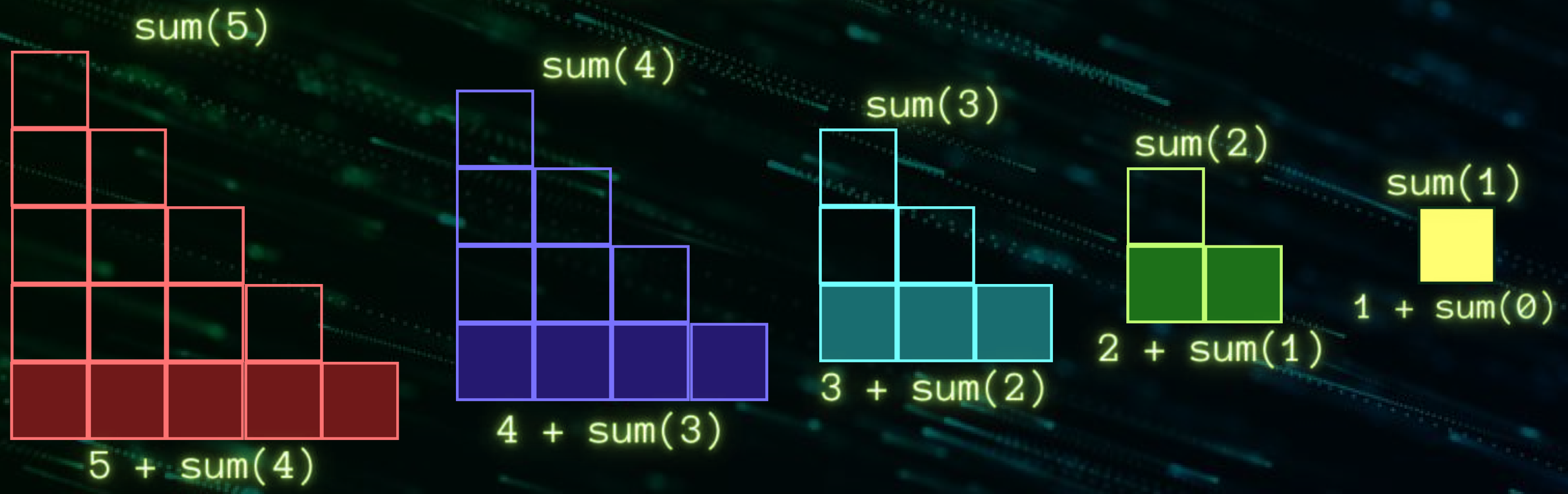
How does the code work?

VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

How does the code work?



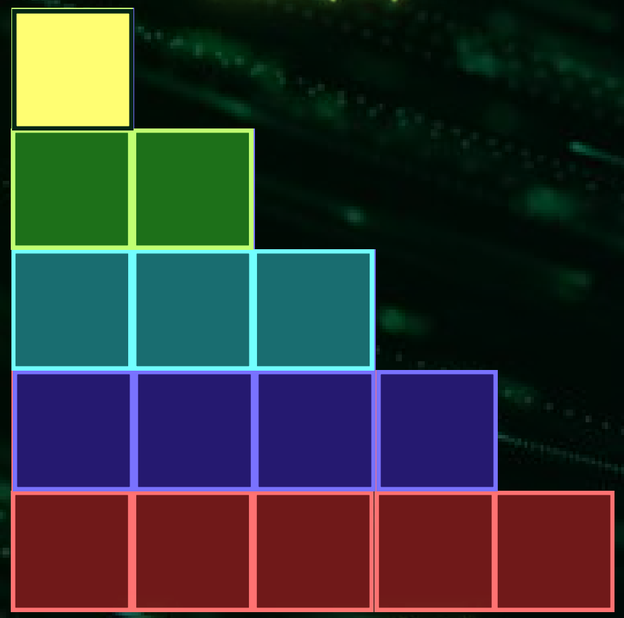
VOUGNE FROID ALIS

EXAMPLE:

Write a recursive function that given an input n sums all nonnegative integers up to n

How does the code work?

sum(5)

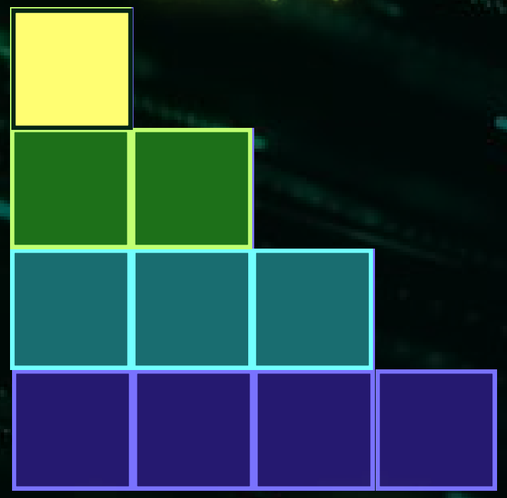


5 + sum(4)

5 + 10

15

sum(4)

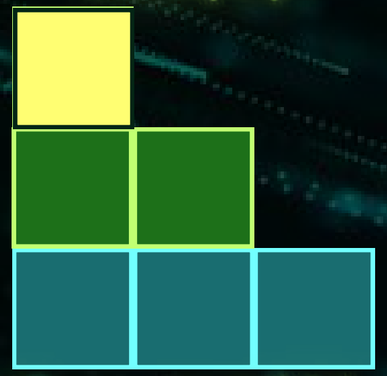


4 + sum(3)

4 + 6

10

sum(3)

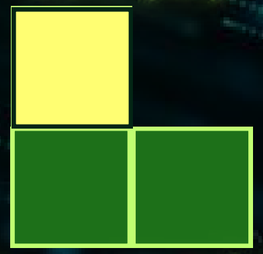


3 + sum(2)

3 + 3

5

sum(2)



2 + sum(1)

2 + 1

3

sum(1)

1 + sum(0)

1 + 0

1

VOUGNE FROID ALIS

RECURSIVE: FACTORIAL

ACTIVITY: FACTORIAL

Similar to the sum but multiplication this time

factorial(0) = 0

factorial(1) = 1

factorial(4) = 1*2*3*4 = 24

factorial(5) = 1*2*3*4*5 = 120

Lets take a look at the iterative approach

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return factorial(3 - 1) * 3
```

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return factorial(3 - 1) * 3
```

2

```
def factorial(2):
    if 2 == 1: return 1
    else: return factorial(2 - 1) * 2
```

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return factorial(3 - 1) * 3
```

2

```
def factorial(2):
    if 2 == 1: return 1
    else: return factorial(2 - 1) * 2
```

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return factorial(3 - 1) * 3
```

2

```
def factorial(2):
    if 2 == 1: return 1
    else: return factorial(2 - 1) * 2
```

1

```
def factorial(1):
    if 1 == 1: return 1 ✓
    else: return factorial(1 - 1) * 1
```

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return factorial(3 - 1) * 3
```

2

```
def factorial(2):
    if 2 == 1: return 1
    else: return 1 * 2
```

```
def factorial(1):
    if n == 1: return 1 ✓
    else: return factorial(1 - 1) * 1
```

VOUGNE FROID ALIS

n = 3

```
def factorial(3):
    if 3 == 1: return 1
    else: return 2 * 3
```

= 6

```
def factorial(2):
    if 2 == 1: return 1
    else: return 1 * 2
```

```
def factorial(1):
    if n == 1: return 1 ✓
    else: return factorial(1 - 1) * 1
```

VOUGNE FROID ALIS

RECURSIVE: FIBONACCI

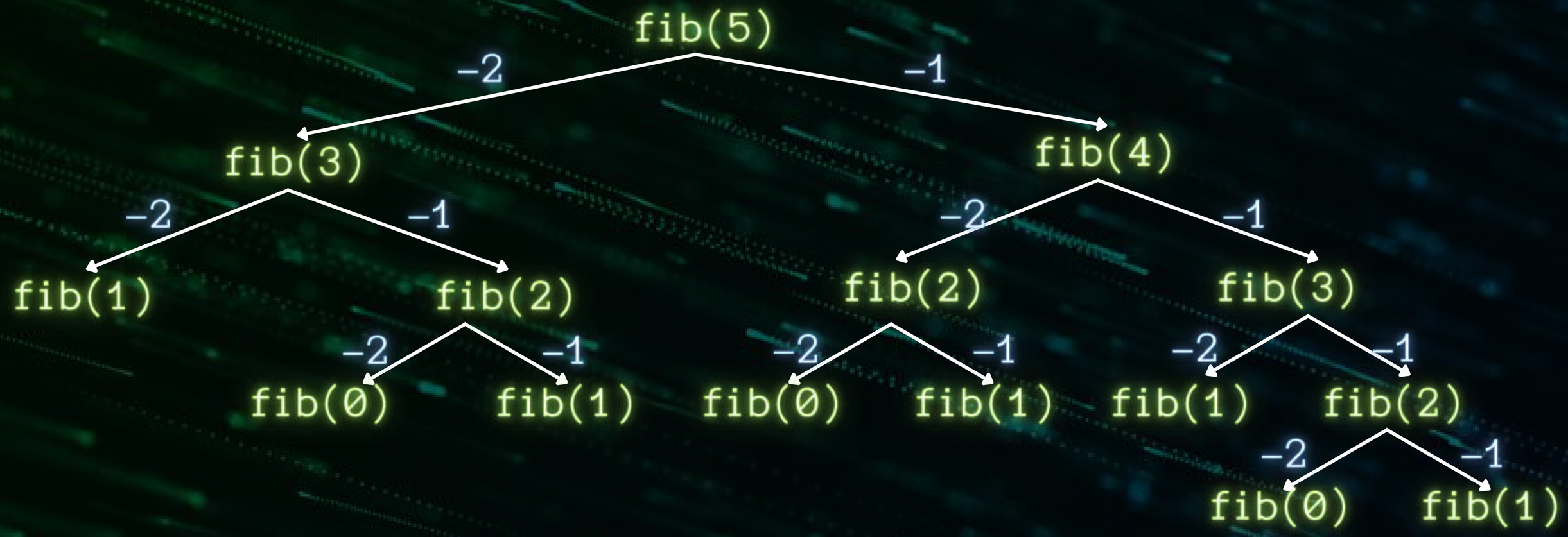
EXAMPLE: FIBONACCI

a series of numbers where each number (Fibonacci number) is the sum of the two preceding ones, usually starting from 0 and 1

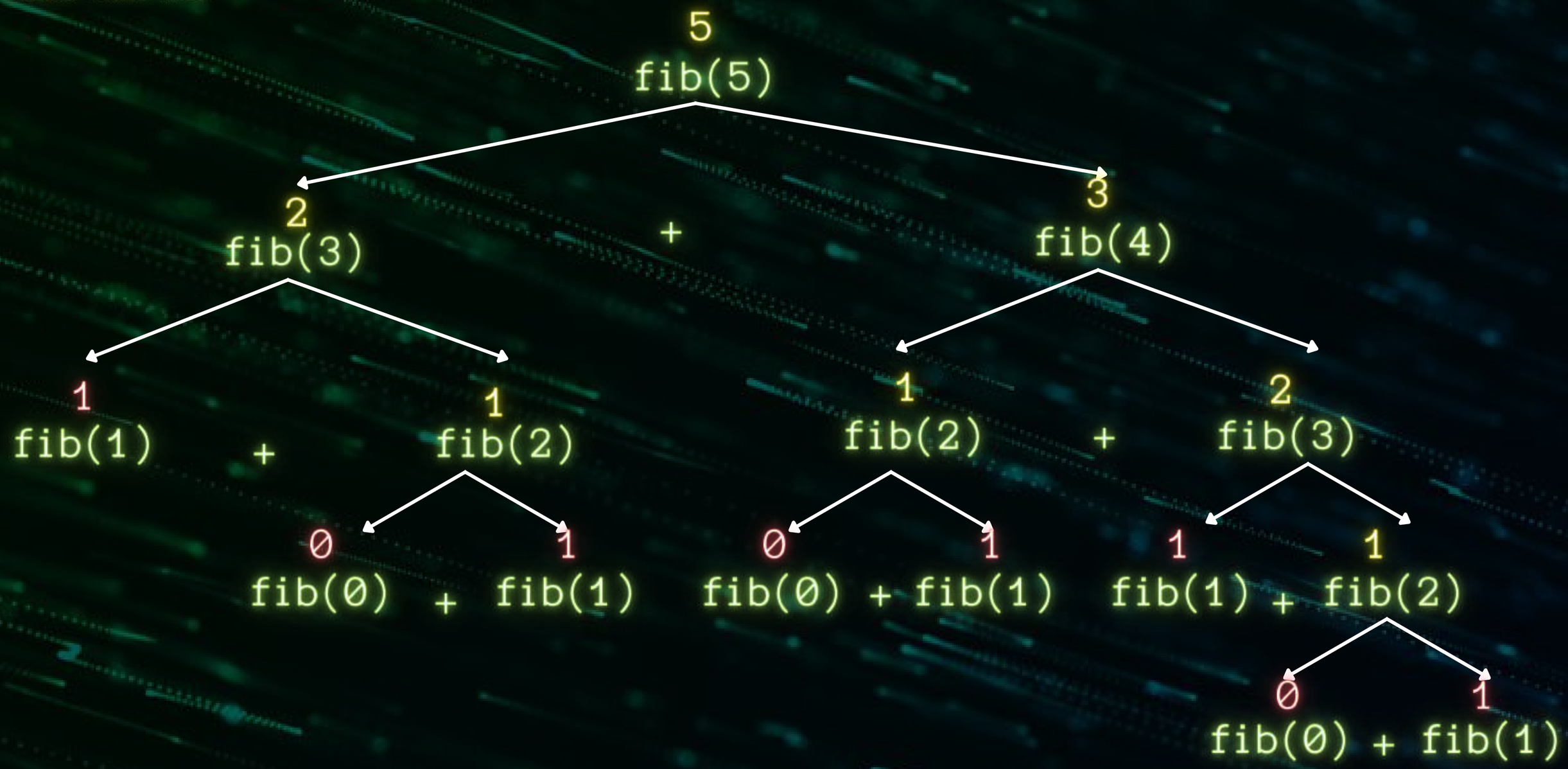
```
fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(4) = 0+1+1+2     = 3
fibonacci(5) = 0+1+1+2+3   = 5
```

Lets take a look at the iterative approach

VOUGNE FROID ALIS



VOUGNE FROID ALIS



VOUGNE FROID ALIS

WHY USE RECURSIVE?

VOUGNE FROID ALIS

WHY USE RECURSIVE?

We use recursion for Trees and Nested Structures

1. The "Unknown Depth" Problem

Imagine you are writing a program to find a specific file on your computer.

- Level 1: C:/
- Level 2: C:/Users/
- Level 3: C:/Users/Von/
- Level 4: C:/Users/Von/Documents/
- ...

You don't know if the file is 3 levels deep or 50 levels

VOUGNE FROID ALIS

WHY USE RECURSIVE?

```
for folder in drive:
    for subfolder in folder:
        for file in subfolder:  # What if the file is deeper?
          if file == "secret.txt":
          print("Found it!")
```

VOUGNE FROID ALIS

WHY USE RECURSIVE?

```python
def find_file(folder, target):
    for item in folder:
        # 1. Base Case: It's a file. Check it.
        if is_file(item):
            if item.name == target:
                print("FOUND IT!")

        # 2. Recursive Case: It's a folder. Dive in!
        elif is_folder(item):
            # This function calls itself to handle the new depth
            find_file(item, target)
```

VOUGNE FROID ALIS

ACTIVITY: STRING REVERSAL

How do you reverse "HELLO" without a loop? You take the last letter 'O' and add it to the reverse of "HELL".

- Recursive Step: Take the last character text and add it to the result of reversing everything else text.
- Base Case: The simplest input is nothing.

VOUGNE FROID ALIS

ACTIVITY: THE PYRAMID BUILDER

Can you print a triangle of stars using recursion? The order of your code lines matters!

- Step 1: Call build_pyramid(n - 1) (Go deeper first!).
- Step 2: Print "*" multiplied by n.
- Base Case: If n is 0, return (stop).

Expected Output:

```
*
**
***
****
```

VOUGNE FROID ALIS

ACTIVITY: NUMBER SUM

Can you calculate the sum of digits (e.g., 452 → 4+5+2 = 11) using only math? No converting to Strings allowed!

- Step 1: Get the Last Digit using modulo (n % 10).
- Step 2: Add that digit to the result of sum_digits called on the Rest of the Number (n // 10).
- Base Case: If n is 0 (no digits left), return 0.

Expected Output:
print(sum_digits(452)) should output 11